

Build Plan — Web, iOS and Android

Dr. Siddharth Gupta Digital Healthcare Platform · companion to PRD v3.0 (Final) ·

Oct 6, 2026 · @RINTU

1. Summary and key decisions

The demo is approved. We build the production platform as **one TypeScript monorepo** that produces a web app, two mobile apps (each running on iPhone, iPad and Android phones and tablets) and one backend. About 70–80% of code — types, validation, business rules, API client, translations, design tokens — is written once and shared. The plan reaches a clinic pilot in about **7 months** with a team of 9, then store launch.

This document is the engineering companion to PRD v3.0 (Final): the PRD says *what* to build; this says *how*, *in what order* and *who*.

#	Decision	Choice
D1	Codebase strategy	One monorepo (pnpm + Turborepo), TypeScript end to end
D2	Web	Next.js app: public website, patient web portal, staff dashboard (doctor, front desk, clinic admin), platform console
D3	iOS and Android	React Native with Expo, one codebase for both platforms and all screen sizes
D4	Number of store apps	Two apps: Patient app and Clinic app (doctor, front desk). Built from the same mobile codebase
D5	iPad and Android tablets	Fully supported in the Clinic app with a two-pane layout; Patient app adapts but is phone-first
D6	Backend	Node.js + Fastify modular monolith, PostgreSQL with row-level security, Redis + BullMQ, S3 — all in AWS Mumbai
D7	API style	REST + OpenAPI 3.1, schemas in Zod, typed client generated for web and mobile
D8	Prescription signing on mobile	Server-side signing with a cloud (remote) Class 3 DSC, authorised by doctor PIN + device biometrics. No USB token needed

#	Decision	Choice
D9	Offline	Read-mostly offline: today's queue, patient summaries and drafts cached on device; writes queued and synced
D10	Releases	EAS Build + Submit for stores, TestFlight and Play testing tracks, web via CI to AWS; over-the-air updates for JavaScript-only fixes
D11	Store accounts	Owned by the legal entity that operates the platform (see open decision O1)
D12	Delivery	6 phases, 2-week sprints, pilot at month 7
D13	Payment gateway	Cashfree Payments on web, iOS and Android: payment links, in-app checkout with UPI Intent, web checkout, refunds, signed webhooks, daily reconciliation (§6.10)

2. Brainstorm: options and trade-offs

Each question below lists the realistic options, what each costs us, and the choice. The deciding factors throughout: a small team, one language across the stack, a doctor who lives on his phone, and a platform that must later serve other doctors.

2.1 How many codebases?

Option	For	Against
A. Fully native (Swift for iOS, Kotlin for Android) + web	Best platform feel and performance	Three codebases and three skill sets; roughly double the mobile team and time
B. Flutter (mobile) + separate web	Good UI performance, one mobile codebase	Dart differs from the TypeScript backend and web; no shared business rules or types with web
C. React Native (Expo) + Next.js in one TypeScript monorepo	One language everywhere; shared validation, safety rules, API client, translations; large hiring pool in India; mature Expo tooling for builds and updates	Native modules occasionally needed (audio, document scanner); heavy tables are better on web than on React Native

Option	For	Against
D. One Expo app that also renders the web (react-native-web)	Maximum sharing	Dense staff tables, keyboard-driven front desk and SEO website suffer; web would feel like a phone app

Choice: C. Mobile and web share packages, not screens. Each surface gets UI that suits it.

2.2 One app or two in the stores?

Option	For	Against
One app with patient and staff modes	One listing to maintain	Mixed sign-in (OTP vs password + 2FA), bigger download for patients, confusing store listing and privacy labels, harder white-labelling
Two apps: Patient app and Clinic app, one codebase	Clear listings and reviews; each app requests only the permissions it needs; Patient app can be white-labelled per organisation later	Two store submissions per release (automated by EAS)

Choice: two apps from one codebase, selected by build profile.

2.3 Where does each staff role live?

- **Doctor:** Clinic app on phone and iPad is primary; web dashboard for long sessions at a desk. Feature parity on both (PRD §3.1).
- **Front desk:** web dashboard on a desktop is primary (keyboard, printer, waiting-room screen); Clinic app on a tablet as a backup.
- **Clinic Admin and Platform Admin:** web only. Settings-heavy work does not justify mobile screens.

2.4 Web framework

Next.js (App Router) for one deployable web app with four areas: public website (server-rendered for SEO), patient portal, staff dashboard, platform console. The demo's React + Tailwind + shadcn components port directly. Alternative considered: separate Vite single-

page apps per area; rejected because the public site needs server rendering and four deployables add overhead.

2.5 Shared UI between web and mobile?

Share **design tokens** (colours, type scale, spacing, radii), icons and copy; build components natively per platform: shadcn/Tailwind on web, a React Native component kit (NativeWind for Tailwind-style classes) on mobile. Cross-platform UI kits that render both were considered; rejected for V1 because they slow the team down on dense web screens.

2.6 Signing prescriptions from a phone

A USB DSC token cannot plug into an iPhone. Options: (a) Aadhaar eSign per prescription — OTP on every prescription, slow at 75 a day and a per-signature fee; (b) **cloud (remote) DSC** held by a licensed provider, signing on the server after the doctor authorises with a PIN and device biometrics; (c) signature image only — legally weak. **Choice: (b)**, with the signing provider behind an adapter so it can change.

2.7 Capturing scribe audio on mobile

Stream audio from the app over a secure WebSocket in short chunks; buffer locally if the network drops and upload when back (PRD §6.3). Background recording is allowed only while the consultation screen is active, which keeps app review simple.

2.8 Offline approach

Full offline sync (local database replicating server data) was considered and rejected for V1: complex conflict handling for clinical records. Instead: cache today's queue, recent patient summaries and in-progress drafts; queue writes (vitals, notes, check-ins) and sync with clear status. Signing and ABDM always require a connection.

2.9 Build, release and updates

Expo Application Services (EAS) builds both apps in the cloud, submits to App Store Connect and Google Play, and ships over-the-air updates for JavaScript-only fixes. Native changes go through full store review. Web deploys from CI on every merge to main after tests pass.

2.10 Who owns accounts, data and code

Apple and Google developer accounts, cloud account, domain and the repository are owned by the legal entity that operates the platform, with the clinic as a customer. If Dr. Gupta's clinic must own them instead, white-labelling for other doctors becomes harder.

Decision O1 in §11.

3. Platforms and roles

Five deliverables run on six device classes. Every role has one primary surface built first and polished most; secondary surfaces reach full parity for daily tasks by launch.

3.1 Deliverables

Deliverable	Built with	Runs on	Users
Web app	Next.js	Desktop and mobile browsers	Public visitors, patients, doctors, front desk, clinic admins, platform admins
Patient app	Expo (React Native)	iPhone, iPad, Android phones and tablets	Patients and family account holders
Clinic app	Expo (React Native), same codebase	iPhone, iPad, Android phones and tablets	Doctors, front desk
Waiting-room display	Web app route in kiosk mode	Any TV with a browser or Android TV stick	Clinic waiting area
Backend API and workers	Node.js + Fastify	AWS Mumbai	All of the above, plus ABDM, WhatsApp, SMS and payment webhooks

3.2 Role-by-surface matrix

Role	Web	iPhone	iPad	Android phone	Android tablet
Patient	Full (portal)	Primary	Supported	Primary	Supported
Doctor	Full	Primary	Primary (two-pane)	Primary	Full (two-pane)
Front desk	Primary	Daily tasks	Full (two-pane)	Daily tasks	Full (two-pane)
Clinic Admin	Primary	—	—	—	—
Platform Admin	Primary	—	—	—	—
Public / pharmacist	Website, booking, verify page	Opens web	Opens web	Opens web	Opens web

"Daily tasks" for front desk on phones = queue, check-in, walk-in registration, payments and the WhatsApp inbox.

3.3 Minimum supported versions

- iOS and iPadOS 16 and later (covers the large majority of active iPhones; confirm against store analytics at launch).
- Android 9 (API 28) and later, targeting the latest Android API level that Google Play requires at submission time.
- Browsers: latest two versions of Chrome, Edge, Safari and Firefox; Chrome on Android and Safari on iOS for the patient portal.

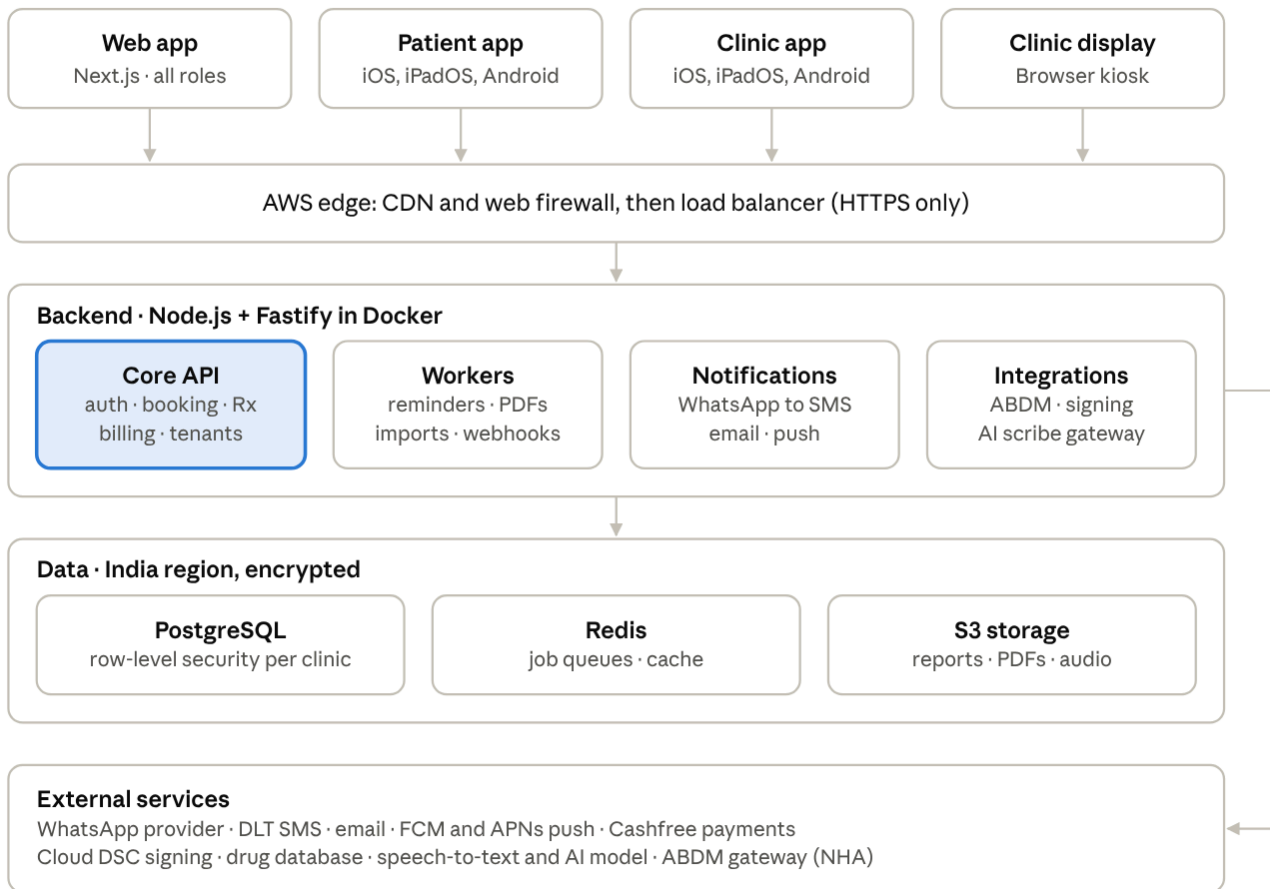
3.4 Screen-size behaviour

- **Phone (compact):** bottom tab bar, single column, actions in bottom sheets.
- **Tablet (regular width) and iPad:** Clinic app switches to two panes — queue or patient list on the left, consultation on the right — and supports landscape, split view and Slide Over on iPad. Keyboard shortcuts on iPad with a keyboard.
- **Web:** sidebar layout for staff; responsive patient portal; public site mobile-first.

4. Architecture and code structure

All clients talk to one API over HTTPS. Business rules — booking, safety checks, signing, tenancy — run only on the server; clients show results. Slow or external work (messages, PDFs, ABDM, imports) runs in background workers so the doctor's screen never waits on a third party.

Four client apps share one backend in AWS Mumbai



system architecture · 4 clients, 1 backend, 3 data stores

The Core API is the one place clinical and billing rules live. Integration services sit behind adapters, so a provider (WhatsApp, SMS, DSC, drug database, AI) can be swapped without touching the rest.

4.1 Backend modules

One deployable service (modular monolith) with strict module boundaries, so any module can be split out later if load demands it.

Module	Owns
identity	Users, sessions, OTP, password + TOTP, devices, biometric unlock tokens
tenancy	Organisations, clinics, branding, settings, row-level security context
scheduling	Availability versions, slot engine, appointments, queue, tokens, visit timing

Module	Owns
patients	Patients, families, tags, UHID, history, allergies, consent
clinical	Consultations, vitals, assessments, scribe sessions, Ask AI
prescribing	Medicine master, safety engine, prescriptions, versions, signing, PDF
orders	Test orders, results, referrals, attachments
billing	Price list, bills, receipts, collections; Cashfree orders, payment links, refunds, webhooks and reconciliation
messaging	Templates, opt-ins, channel cascade, inbox, delivery log
abdm	ABHA, Scan & Share, care contexts, consents, external records
platform	Doctor onboarding, verification, usage metering, import jobs
audit	Append-only audit log, data-rights requests

4.2 Monorepo layout

repo/	
apps/	
web/	Next.js: site, patient portal, staff dashboard, platform
console, display	
mobile/	Expo: Patient app and Clinic app (two build profiles)
api/	Fastify API and module code
workers/	BullMQ workers (same code as api, separate process)
packages/	
contracts/	Zod schemas and OpenAPI spec (single source of truth)
api-client/	Typed client generated from OpenAPI (web and mobile)
domain/	Pure rules: slot engine, dosage remarks, UHID, status
machine	
safety/	Safety-engine rules and test cases
i18n/	English and Hindi strings, date and number formatting
tokens/	Design tokens: colours, type, spacing, radii
ui-web/	shadcn and Tailwind components
ui-native/	React Native components (NativeWind)
config/	ESLint, TypeScript, Prettier, test presets
infra/	Terraform for AWS (network, RDS, ECS, S3, CloudFront, WAF)
docs/	ADRs, runbooks, API guide (see section 11)

The shared rule packages (domain, safety) run on the server as the authority. The apps import the same code only for instant feedback, such as a dosage sentence or a warning preview, before the server confirms.

4.3 API conventions

- REST over HTTPS, versioned under /v1, JSON. OpenAPI 3.1 generated from the Zod schemas in packages/contracts.
- Idempotency keys on every write that mobile may retry after being offline (check-in, vitals, payments, notes).
- Cursor pagination; dates stored in UTC, shown in IST.
- One error shape (code, message, field errors) so web and mobile handle errors the same way.
- Incoming webhooks (WhatsApp, SMS, payment, ABDM) are signature-verified and processed by workers.
- Live queue updates over Server-Sent Events on web and WebSocket on mobile; scribe audio uses its own WebSocket.

4.4 Data and tenancy

- PostgreSQL 16 on RDS, Multi-AZ in production. Every table carries organisationId; row-level security policies enforce isolation, and each request sets the tenant context from the user's token.
- Prisma for schema and migrations; hand-written SQL allowed for reports, with review.
- S3 buckets per environment: private, encrypted, objects addressed by key and served through short-lived signed URLs (PRD §11.2).
- Field-level encryption for clinical notes, diagnoses, transcripts and signing authorisation material.

5. Platform specifics

The shared core is the same everywhere; this section lists what each platform needs beyond it.

5.1 Web (Next.js)

Area	Route group	Rendering	Notes
Public website	/, /doctor/[slug], /verify	Server-rendered, cached at the CDN	Per-organisation branding and custom domain; SEO metadata and doctor schema markup
Patient portal	/app	Client-rendered behind login	Booking, records, prescriptions, payments, consents; works on mobile browsers for patients without the app
Staff dashboard	/clinic	Client-rendered behind login	Doctor, front desk and clinic admin; keyboard shortcuts; print layouts for prescriptions, receipts and day sheets
Platform console	/platform	Client-rendered, separate login and IP allow-list	Organisations, doctor verification, usage, health
Waiting-room display	/display/[clinic]	Kiosk mode, read-only token	Current and next tokens only; no names

- Web scribe uses the browser microphone (MediaRecorder) with the same streaming protocol as mobile.
- Printing: browser print with dedicated A4/A5 CSS for prescriptions and receipts; thermal receipt printer support is a Phase 2 option.
- Installable as a Progressive Web App for front-desk PCs so it opens like a desktop app.
- Security headers: strict Content Security Policy, HSTS, no third-party scripts on clinical pages.

5.2 iOS and iPadOS

- **Two bundle IDs:** Patient app and Clinic app, each with its own App Store listing, icons and privacy details.
- **iPad (Clinic app):** two-pane layout, landscape and portrait, Split View and Slide Over, external keyboard shortcuts (next patient, sign, search), pointer support. Patient app runs full-screen on iPad with adaptive layouts.
- **Permissions requested only when used, with clear purpose strings:** microphone (scribe), camera (documents, ABDM QR), photo library (uploads), notifications, Face ID (unlock and signing approval).
- **Push:** APNs through Firebase Cloud Messaging; notification categories with action buttons (Confirm, Reschedule) on the Patient app.
- **Deep links:** Universal Links for booking, prescription, payment and consent links sent by WhatsApp and SMS.
- **Secure storage:** tokens in the Keychain; Face ID or passcode to reopen the Clinic app after 15 minutes idle.
- **Files:** document scanner for multi-page reports, PDF viewer, share sheet, AirPrint for prescriptions.
- **Background:** scribe recording continues only while the consultation screen is open; uploads finish with background tasks.
- **Privacy:** App Store privacy labels and a privacy manifest declaring data use; in-app account deletion (Apple requirement).

5.3 Android (phones and tablets)

- **Two application IDs,** matching the iOS split, published under one Google Play organisation account.
- **Tablets and foldables (Clinic app):** two-pane layout on large screens, resizable windows, keyboard shortcuts.
- **Permissions at runtime** with rationale screens: microphone, camera, notifications (Android 13+), media access via the system photo picker where possible.

- **Push:** Firebase Cloud Messaging with notification channels (Appointments, Prescriptions, Payments, Security) so patients can mute categories.
- **Deep links:** verified App Links for the same link types as iOS.
- **Secure storage:** Android Keystore; biometric prompt for unlock and signing approval.
- **Scribe:** foreground service with a visible notification while recording, as Android requires for microphone use in the background.
- **Device range:** test on low-memory devices common in India (2–3 GB RAM); keep the Patient app download small; aggressive image compression before upload.
- **Store:** Data safety form, health apps declaration, target the API level Google Play currently requires, app bundles (AAB) only.

5.4 What is shared vs per platform

Layer	Shared across web, iOS, Android	Per platform
Types, validation, API client	Yes	—
Business rules (slot engine, dosage remarks, safety previews)	Yes	—
Translations, date/number formats	Yes	—
Design tokens, icons	Yes	—
Screens and components	Mobile screens shared by iOS and Android	Web screens separate
Navigation	Expo Router (mobile)	Next.js routing (web)
Device features	—	Camera, mic, biometrics, push, file system, printing

6. Cross-cutting engineering

These features touch every platform and carry the most risk, so each gets a named owner and a written design (ADR) before its sprint starts.

6.1 Sign-in and sessions

Who	First sign-in	Returning	Session
Patient	Mobile OTP by SMS (WhatsApp OTP optional)	Biometric unlock on device; OTP on web	Refresh token rotated, up to 30 days on a trusted device
Doctor, front desk	Password + TOTP authenticator, device registered	Face ID / fingerprint on registered device; password + TOTP on new devices and web	15-minute idle lock on mobile, 15-minute idle logout on web
Clinic and platform admins	Password + TOTP	Same, web only	Short sessions; platform console IP allow-list

Lost device: staff can revoke a device from the web; tokens on that device stop working immediately.

6.2 Prescription signing

1. Doctor taps Sign on any device.
2. App asks for biometric confirmation (or signing PIN on web).
3. Server validates the prescription again (safety engine, completeness, verified doctor) and generates the PDF.
4. Server asks the cloud DSC provider to sign the PDF hash using the doctor's certificate.
5. Signed PDF and its hash are stored; prescription is locked; delivery starts.

The doctor's certificate never leaves the provider. Every signing event is in the audit log with device and time.

6.3 AI scribe pipeline

- Client captures audio in short chunks (about 1 second), streams over an authenticated WebSocket to the scribe gateway, and buffers locally when offline.
- Gateway forwards to the speech-to-text provider, assembles the transcript with speaker labels, then asks the language model for the structured draft using a versioned prompt.
- Drafts return to the consultation screen; the doctor accepts section by section (PRD §6.3).
- Audio stored encrypted in S3 with automatic deletion after the retention window; transcripts kept with the consultation.

- Provider adapters let us test several vendors on the same recorded (consented, dummy) sessions before choosing.

6.4 Offline and poor networks

Works offline	Queued until online	Needs a connection
Today's queue, patient summaries already opened, drafts, templates, medicine master	Check-in, vitals, notes, tags, counter payments, scribe audio upload	Booking new slots, signing, payment links, ABDM, Ask AI, safety checks against the live database

Queued writes carry idempotency keys and show a clear "waiting to sync" badge. If the server rejects a queued change (for example, a slot already taken), the app explains and offers the fix.

6.5 Notifications and deep links

- One notification service decides the channel (WhatsApp, SMS, email, push) per PRD §8.1; apps only register devices and handle taps.
- Every link in a message is a signed, short-lived deep link that opens the app if installed, otherwise the web portal with OTP or date-of-birth check.

6.6 Files and documents

- Uploads go directly from the device to S3 using pre-signed URLs, then the server scans for malware before the file becomes visible.
- Images compressed on device; multi-page photos combined into one PDF on device or by a worker.
- Generated documents (prescriptions, receipts, referral letters) are rendered server-side from versioned HTML templates.

6.7 Multi-tenant branding and white-label

- Web: each organisation gets a subdomain or custom domain; theme, logo and text load at request time.
- Mobile V1: one Patient app and one Clinic app; branding (logo, colours, display name) loads after sign-in or from a clinic invite link.
- Phase 2: white-label Patient app builds per organisation from the same codebase using EAS build profiles (own name, icon and store listing).

6.8 Accessibility and language

- WCAG 2.1 AA on web; Dynamic Type (iOS) and font scaling (Android) supported; screen-reader labels on every control; 44 px minimum touch targets.
- English and Hindi at launch across all apps; strings live only in packages/i18n so more languages are a translation task, not a code change.

6.9 Observability

- Structured logs without patient data; error tracking on web, mobile and backend with release tags; uptime checks on the API and website.
- Mobile: crash reporting, app start time, screen load time, offline-queue size.
- Business dashboards: bookings, completed visits, prescriptions, message delivery and cost, scribe minutes — per organisation.

6.10 Payments with Cashfree

All online money movement goes through Cashfree Payments, behind a payments adapter in the billing module so the rest of the code never calls Cashfree directly. Product behaviour is specified in PRD §5.6; this section covers how it is built.

Surface	Integration	Used for
Backend	Cashfree server SDK for Node.js / REST API: create orders, create and cancel payment links, fetch order status, refunds, settlements	Source of truth for every amount and status
Patient app (iOS, Android)	Cashfree React Native SDK; UPI Intent flow so GPay, PhonePe, Paytm and other UPI apps open directly, plus card and net banking	Pay at booking, pay dues
Web (patient portal, website booking widget)	Cashfree JavaScript checkout	Pay at booking, pay dues
Clinic app and web dashboard	No SDK; staff create bills and send payment links through our API	Payment links, refunds
Webhooks	Cashfree payment, refund and settlement webhooks to a dedicated endpoint	Confirming payments and refunds

Payment flow, step by step:

1. App asks our API to pay a bill; the API creates a Cashfree order for the exact bill amount (idempotency key = bill id + attempt) and returns a payment session.
2. App opens Cashfree's checkout (SDK on mobile, JS checkout on web). Card and UPI details never pass through our servers.
3. On return, the app shows "Confirming payment..." and asks our API for status; the API checks the order with Cashfree.
4. Cashfree's webhook arrives; the API verifies the signature and timestamp, stores the event, de-duplicates it, and marks the bill Paid. Receipt and messages are sent by workers.
5. If neither webhook nor status check confirms within the hold time, the attempt is marked Failed or Expired and a booking hold is released.

Build notes:

- The React Native SDK is a native module, so the apps use Expo development builds and EAS builds (not Expo Go). UPI apps must be declared for app detection as Cashfree's guide requires (URL schemes on iOS, package visibility queries on Android).

- Per-organisation Cashfree keys are stored encrypted in the secrets store; each request picks the organisation's keys from the tenant context.
- Webhook endpoint is public but accepts only signed Cashfree events; it responds quickly and hands work to a worker queue.
- Nightly reconciliation job pulls settlements and refunds, matches them to bills, and raises mismatches on the Clinic Admin dashboard.
- Sandbox keys and Cashfree test payment methods in dev and staging; production keys only in production.
- Tests: unit tests for amount and status logic, contract tests against Cashfree sandbox, and end-to-end tests that pay with sandbox UPI and card on iOS, Android and web.

Cashfree onboarding checklist:

- ☐ Merchant account for Gupta Clinic (business KYC, settlement bank account) — start in week 1, as activation can take time.
- ☐ Enable UPI, cards, net banking and wallets; agree transaction fees.
- ☐ Generate sandbox and production API keys; register webhook URLs for staging and production.
- ☐ Confirm website and app details Cashfree requires for activation (domain, policies, refund policy, contact page).
- ☐ Settlement schedule agreed and reflected in the reconciliation report.

7. Environments, CI/CD and releases

Every change reaches users the same way: pull request, automated checks, staging, then production. Mobile adds store review and staged rollouts on top.

7.1 Environments

Environment	Purpose	Data	Mobile builds
Local	Developer machines; Docker for Postgres and Redis	Seed data	Expo development builds
Dev	Every merge to main deploys automatically	Synthetic data, reset weekly	Internal distribution builds
Staging	Release candidates, QA, clinic user acceptance, store review accounts	Synthetic data only, never real patients	TestFlight and Play internal / closed testing

Environment	Purpose	Data	Mobile builds
Production	Live clinic	Real patient data, India region	App Store and Play production, staged rollout

Providers run in sandbox mode outside production: ABDM sandbox, Cashfree sandbox, WhatsApp test number, DSC test certificates.

7.2 Continuous integration (every pull request)

1. Type check, lint, format.
2. Unit tests (domain, safety, api modules) and component tests.
3. Contract check: OpenAPI spec regenerated; API client must compile in web and mobile.
4. Database migration dry-run against a copy of the staging schema.
5. Tenant-isolation tests: requests from organisation A must never return organisation B's data.
6. Security scans: dependencies, secrets, container images.
7. Preview deployment of the web app for review.

Merge needs one approving review and green checks; changes to safety rules, signing, tenancy or auth need a second reviewer.

7.3 Web and backend release

- Container images built once, promoted from dev to staging to production unchanged.
- Database migrations run before the new version, written to be backward compatible (expand, then contract) so a rollback never breaks.
- Blue-green deploy on ECS; automatic rollback if health checks or error rates fail.
- Release cadence: weekly to production after staging sign-off; hotfixes any day.

7.4 Mobile release

- EAS Build produces iOS and Android binaries for both apps from one commit; EAS Submit uploads them to App Store Connect and Google Play.
- Testing tracks: TestFlight (internal, then the clinic staff as external testers) and Google Play internal then closed testing.
- Production: Play staged rollout (10% → 50% → 100%) and App Store phased release over 7 days, watching crash rate and errors.

- Over-the-air updates (EAS Update) only for JavaScript and asset fixes that do not change app behaviour beyond what was reviewed; native or permission changes always go through store review.
- Cadence: store release every 2 weeks during build, monthly after launch; API stays backward compatible for at least the two latest app versions, with a forced-update screen for anything older.

7.5 Versioning and change records

- Semantic versions for apps; build numbers from CI.
- Release notes generated from merged pull requests and reviewed by the product owner.
- Every production release recorded with version, date, migration list and approver (needed for audits).

8. Quality, testing and devices

Clinical software is judged by its worst bug, so testing effort follows risk: the safety engine, signing, tenancy and billing get the most.

8.1 Test layers

Layer	Tool	What it covers	Gate
Unit	Vitest	Domain rules, safety rules, dosage sentences, slot engine, UHID	Every PR
API integration	Vitest + test database	Each endpoint with roles, tenants and error cases	Every PR
Tenant isolation	Dedicated suite	Cross-organisation reads and writes through every endpoint and file link	Every PR; blocks merge
Web end-to-end	Playwright	Booking, consultation to sign, front desk day, admin settings	Every PR (smoke), nightly (full)
Mobile end-to-end	Maestro	Same journeys on iOS simulator and Android emulator, phone and tablet	Nightly and before each store build

Layer	Tool	What it covers	Gate
Clinical safety	Rule test pack	Every rule in PRD §6.5 with positive and negative cases, signed off by a clinician	Any change to safety rules
Accessibility	axe (web), screen-reader passes (mobile)	Contrast, labels, focus, Dynamic Type	Before each release
Performance	k6 load tests, app profiling	API at 10× expected peak; app start and list scrolling on low-end Android	Before pilot and launch
Security	External penetration test	Web, both apps, API, tenant isolation	Before pilot; yearly

8.2 Device matrix

A physical device lab of about eight devices, plus a cloud device service for wider coverage before each release.

Class	Minimum to test
iPhone	One small screen (SE size), one current Pro Max size, oldest supported iOS and latest iOS
iPad	One iPad (10th gen class) and one iPad Pro, with keyboard, both orientations, Split View
Android phone, low end	2–3 GB RAM device on Android 9–11
Android phone, mid range	Popular Indian brand model on recent Android
Android tablet	One 10–11 inch tablet
Web	Chrome and Edge on Windows (front desk PCs), Safari on Mac, mobile browsers

8.3 Performance budgets

- Patient app cold start under 3 s on a mid-range Android; Clinic app queue visible under 1.5 s from unlock on a warm start.
- API p95 under 400 ms; prescription PDF under 5 s; scribe draft under 30 s for a 10-minute consultation.

- Patient app download under 40 MB on Android.

8.4 Clinical acceptance

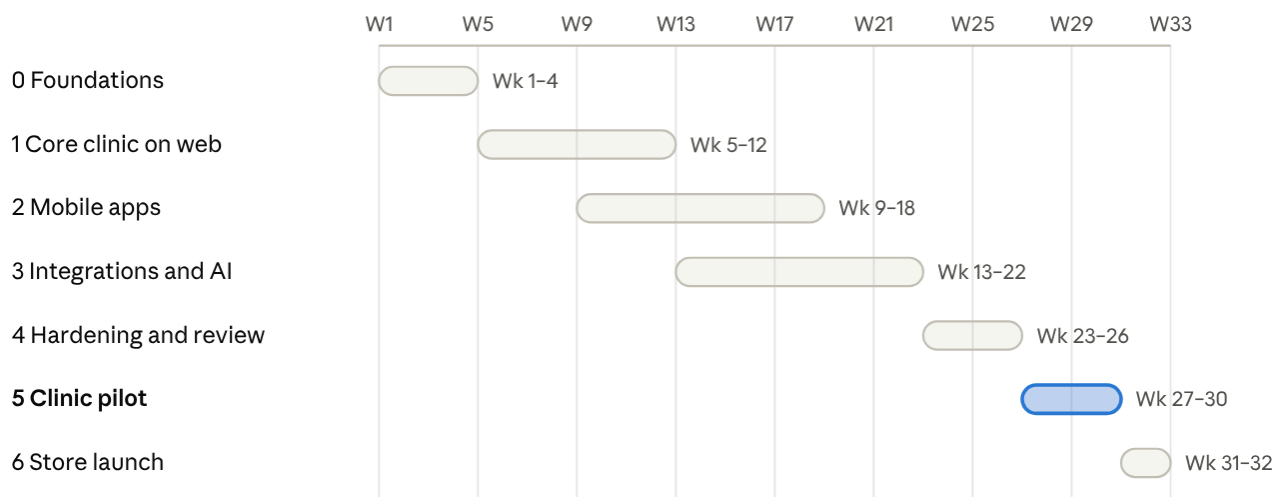
Before pilot, Dr. Gupta and front-desk staff run a full simulated clinic day on staging (dummy patients) across web, iPhone, iPad and Android, using the acceptance criteria in PRD v3.0 §12.2. Issues found are triaged as blocker, must-fix before launch, or later.

9. Delivery plan

Web comes first because it unblocks the clinic's front desk and gives the doctor a full workflow early; mobile starts four weeks later on the same API; integrations run alongside. The pilot uses all three platforms at once.

Clinic pilot starts in week 27, store launch around week 31

Phases by week from project start; phases 1 to 3 overlap by design



delivery timeline · 7 phases over 32 weeks

Week 1 starts when the team is hired and the open decisions in §11 are closed. The overlap between phases 1, 2 and 3 is what keeps the plan at about 7 months to pilot; a smaller team removes the overlap and adds 2 to 3 months.

9.1 Phases and exit criteria

Phase	Weeks	Scope	Exit criteria
0 Foundations	1–4	Monorepo, CI/CD, AWS dev and staging, design tokens, auth (OTP, password + TOTP), tenancy with row-level security, audit log, ADRs; start DLT, WhatsApp, Apple, Google, ABDM sandbox, DSC provider and Cashfree merchant KYC paperwork; choose drug database and AI vendors	A logged-in user in organisation A cannot see organisation B; CI green; accounts applied for
1 Core clinic on web	5–12	Scheduling and slot engine, patients, families, tags, UHID, queue tabs, front desk, consultation, vitals, prescription builder, safety engine, cloud DSC signing, PDF, billing and counter payments, clinic admin settings	A full simulated clinic day runs on the web dashboard
2 Mobile apps	9–18	Clinic app (phone and iPad/tablet two-pane, biometrics, offline queue) and Patient app (onboarding, family, booking, records, prescriptions, consents) on iOS and Android	Both apps on TestFlight and Play internal testing; doctor completes a day on iPad and phone
3 Integrations and AI	13–22	Messaging cascade and inbox, Cashfree payments (links, in-app and web checkout, refunds, reconciliation), ABDM M1, Scan & Share and M2, AI scribe, Ask AI, assessments, orders, referrals, review links, multi-doctor onboarding and platform console, import tool	Every PRD §11.2 acceptance criterion passes on staging

Phase	Weeks	Scope	Exit criteria
4 Hardening and review	23–26	Penetration test and fixes, load test, accessibility, ABDM review submission, trial migration from the current EMR, store listings and review	No open critical or high findings; store builds approved; migration dry run clean
5 Clinic pilot	27–30	Production go-live at the clinic with real patients; parallel run with the current EMR for 2 weeks, then cut-over	Clinic runs fully on the platform for 2 weeks with no blocker issues
6 Store launch	31–32	Public App Store and Play release, patient onboarding campaign at the clinic, ABDM M3 (fetch records) in the following weeks	Apps live; first onboarded patients using booking and prescriptions

9.2 Sprint rhythm

- Two-week sprints (16 sprints to launch): planning on day 1, demo to Dr. Gupta every second Friday, retrospective at the end.
- Each sprint ships to staging; a store build goes to testers every sprint once phase 2 starts.
- A definition of done applies to every feature: works on web and both mobile platforms where the role matrix says so, tests written, translations added, audit events logged, documentation updated.

9.3 Team

Role	Count	Main responsibilities
Tech lead / architect	1	Architecture, ADRs, code review of risky areas, security owner
Backend engineers	2	API modules, workers, integrations (ABDM, messaging, payments, DSC, AI)
Web engineer	1	Next.js app: staff dashboard, patient portal, website, platform console
Mobile engineers	2	Expo apps for iOS and Android, tablets, offline, device features, store releases

Role	Count	Main responsibilities
Product designer	1	Design system, flows for all platforms, usability sessions with the clinic
QA engineer	1	Test plans, automation (Playwright, Maestro), device lab, release sign-off
DevOps (part-time)	0.5	AWS, Terraform, CI/CD, monitoring, backups
Product owner / project manager	1	Backlog, vendor coordination, compliance tracking, clinic liaison

Advisors on call: a clinician to review safety rules and the scribe, and a healthcare lawyer for consent, DPDP and ABDM documents.

9.4 Who decides what

Decision	Responsible	Approves	Consulted
Scope and priorities	Product owner	Platform owner (you)	Dr. Gupta
Architecture and vendors	Tech lead	Platform owner	Engineers
Clinical rules and templates	Product owner	Dr. Gupta	Clinical advisor
Release to production	QA engineer	Tech lead	Product owner
Security and privacy exceptions	Tech lead	Platform owner	Lawyer

10. Store readiness, costs and risks

App review for healthcare apps is stricter than for ordinary apps, and the paperwork has long lead times, so it starts in week 1, not at the end.

10.1 App Store (Apple) checklist

- ☐ Apple Developer Program enrolled as an **organisation** (needs a D-U-N-S number). Apple's guidelines expect apps in regulated fields such as healthcare to be submitted by the legal entity providing the service, not an individual.
- ☐ Two app records (Patient, Clinic) with names, icons, screenshots for iPhone and iPad, and support and privacy-policy URLs.
- ☐ Privacy labels and privacy manifest completed for each app.
- ☐ In-app account deletion available in the Patient app.

- ☐ Review notes: demo accounts for patient and doctor on staging-like review data, explanation of telemedicine, OTP bypass for the reviewer number, and how the scribe consent works.
- ☐ Microphone, camera and photo purpose strings written in plain language.
- ☐ Export compliance answers (standard encryption only).

10.2 Google Play checklist

- ☐ Play Console **organisation** account (D-U-N-S verification); Google increasingly expects health apps to be published by organisations.
- ☐ Two app listings with phone and tablet screenshots.
- ☐ Data safety form and health apps declaration for each app.
- ☐ Target API level as required by Play at submission time; app bundles signed with Play App Signing.
- ☐ Foreground-service declaration for microphone use (scribe) in the Clinic app.
- ☐ Account deletion link (in-app and web) declared.
- ☐ Closed testing with clinic staff before production.

10.3 Costs

One-off and yearly items below are known list prices or vendor quotes still to collect. Team cost is a rough planning estimate for India and should be replaced by real quotes.

Item	Type	Estimate
Team of about 9 for 8 months (in-house or agency)	Build	Rough planning range ₹1-1.5 crore; get two or three quotes
Apple Developer Program	Yearly	US\$99
Google Play developer account	One-off	US\$25
Expo EAS paid plan (cloud builds, submit, updates)	Monthly	Check current pricing; optional if builds run on our own CI
Mac build machine or cloud Mac minutes (iOS builds)	One-off or monthly	Covered by EAS cloud builds if used
Physical device lab (about 8 devices)	One-off	Approx. ₹3-5 lakh
Cloud device testing service	Monthly	Approx. ₹10,000-25,000

Item	Type	Estimate
AWS (dev, staging, production)	Monthly	Approx. ₹20,000–40,000 during build; production as in the cost note
Penetration test	Before pilot, then yearly	Approx. ₹1–3 lakh for web, two apps and API
Drug database licence, AI scribe usage, WhatsApp, SMS, Cashfree transaction fees, cloud DSC	Monthly / per use	Vendor quotes during phase 0
Legal review (consent, privacy, ABDM, AI)	One-off	Approx. ₹50,000–1.5 lakh

10.4 Risks

Risk	Likelihood	Impact	Mitigation
App review rejection (health data, telemedicine, scribe)	Medium	Launch slips 1–3 weeks	Organisation accounts, clear review notes, submit a TestFlight external build early to surface issues
ABDM review takes longer than planned	Medium	ABDM features launch late	Start sandbox in week 1; launch with ABDM behind a feature flag
Cloud DSC provider integration harder than expected	Medium	Signing blocked	Shortlist two providers in phase 0; keep the signing adapter thin; fallback to visible-signature mode only in staging
Mobile team stretched across two apps and four device classes	High	Quality drops on tablets	Shared component kit, tablet layouts designed up front, Maestro tests on tablets every night
Low-end Android performance	Medium	Patient drop-off	Performance budget in CI, testing on 2–3 GB RAM devices from sprint 1
Offline sync conflicts	Low	Duplicate or lost entries	Limited offline scope (§6.4), idempotency keys, clear sync status

Risk	Likelihood	Impact	Mitigation
Data migration from current EMR is partial	Medium	Doctor lacks history	Request export early; fallback plan in PRD §9.5
Hiring delays	Medium	Whole plan shifts	Start with tech lead and one mobile and one backend engineer; agency for overflow

11. Documentation, open decisions and next steps

"Document everything" means a small set of living documents with named owners, kept next to the code where possible, so they stay true as the product changes.

11.1 Documentation set

Document	Owner	Lives in	Updated when
PRD (v3.0 Final and later)	Product owner	Claude Docs	Scope changes
Build plan (this document)	Tech lead	Claude Docs	End of each phase
Architecture decision records (ADRs), one per decision D1-D12 and each new one	Tech lead	docs/adr in the repo	Before a decision is implemented
API reference	Generated	OpenAPI spec, published to the developer portal	Every merge
Data dictionary	Backend lead	docs/data in the repo	Every migration
Safety rule catalogue with clinical sign-off	Product owner + clinical advisor	docs/safety	Any rule change
Security and privacy pack (threat model, data flows, DPDP record of processing, vendor DPAs)	Tech lead + lawyer	Shared drive, restricted	Quarterly and on vendor change

Document	Owner	Lives in	Updated when
Test plan and release checklist	QA engineer	docs/qa	Each release
Runbooks (deploy, rollback, incident, backup restore, key rotation, store submission)	DevOps	docs/runbooks	After every incident or drill
Store submission pack (listings, screenshots, review notes, privacy answers)	Product owner	Shared drive	Each store release
User guides: doctor, front desk, clinic admin, patient FAQ	Product designer	Help centre in the web app	Each feature release
Release notes and change log	Product owner	Web app and store listings	Each release

Rule: a pull request that changes behaviour updates the relevant document in the same pull request.

11.2 Open decisions to close in week 1

- ☐ **O1** Which legal entity owns the Apple and Google accounts, AWS, domain and code (your company vs the clinic). Affects white-labelling for other doctors.
- ☐ **O2** Build team model: in-house hires, agency, or a mix.
- ☐ **O3** Cloud DSC provider (shortlist two).
- ☐ **O4** Drug database and AI scribe vendors (from PRD open decisions).
- ☐ **O5** WhatsApp provider and SMS provider. Payment gateway decided: Cashfree.
- ☐ **O6** Whether front desk needs the Clinic app at launch or web only for V1.
- ☐ **O7** Confirm Dr. Gupta's specialty pack (orthopaedics) and pilot start date.

11.3 Next steps (first two weeks)

1. Close O1 and O2; start hiring or agency selection with this document and PRD v2.2 as the brief.
2. Apply for D-U-N-S, Apple and Google organisation accounts, DLT registration, WhatsApp Business verification and ABDM sandbox access — these have the longest lead times.

3. Request a data export from the current EMR vendor.
4. Tech lead writes ADRs for D1–D12 and sets up the monorepo, CI and dev environment.
5. Designer turns the approved demo into the production design system for web, phone and tablet.
6. Book the first demo with Dr. Gupta for the end of sprint 2.